

Análise de Código Java

Padrão Circuit Breaker

■ Excerto do Código

Núcleo da lógica de transição de estados:

```
enum State { CLOSED, OPEN, HALF_OPEN }

public String call(Supplier<String> action) {
    if (state == State.OPEN) {
        if (now - openedAt > cooldownMs)
            state = State.HALF_OPEN;
        else return "BLOCKED";
    }
    try {
        String result = action.get();
        if (state == State.HALF_OPEN)
            { state = State.CLOSED; failures = 0; }
        return result;
    } catch (Exception e) {
        failures++;
        if (failures >= threshold)
            { state = State.OPEN; openedAt = now; }
        return "FAILED: " + e.getMessage();
    }
}
```

■ Explicação Detalhada

A classe **CircuitBreaker** implementa o padrão de resiliência homónimo, cujo objectivo é proteger um sistema contra falhas em cascata ao interromper chamadas a serviços instáveis. O funcionamento baseia-se numa máquina de estados com três modos: **CLOSED** (operação normal), **OPEN** (chamadas bloqueadas após atingir o limiar de falhas) e **HALF_OPEN** (período de sondagem após o cooldown, para verificar se o serviço recuperou). O método `call()` recebe um **Supplier<String>** — interface funcional do Java — e executa-o dentro de um bloco try/catch: em caso de excepção, o contador de falhas é incrementado e, ao atingir o *threshold*, o circuito abre e regista o instante (*openedAt*). Após o período de cooldown (verificado via `System.currentTimeMillis()`), o estado transita para **HALF_OPEN** e a próxima chamada bem-sucedida fecha o circuito novamente. O design é simples e não é *thread-safe*: em ambientes concorrentes seria necessário sincronização (e.g., `AtomicInteger`, `ReentrantLock`). A complexidade temporal é $O(1)$ por chamada, sem estruturas de dados adicionais.

■ Resumo Técnico

Objectivo	Proteger serviços contra falhas em cascata
-----------	--

Padrão	Circuit Breaker (Resiliência)
Conceitos Java	Enum, Supplier<T>, interface funcional, try/catch
Estados	CLOSED → OPEN → HALF_OPEN → CLOSED
Complexidade	O(1) por chamada — sem estruturas auxiliares
Limitação principal	Não é thread-safe (sem sincronização)